

航空应急仿真培训平台

详细项目实施方案（终极架构版）

当前交付目标：气囊顶升 Web 培训模块

长期扩展方向：多机型、多场景、灭火 VR、力反馈与实体操作设备

建议首期周期：16 周（约 4 个月）

版本：V1.0

1. 项目定位与实施原则

项目定位：本项目不是单一“气囊网页”，而是一套可长期扩展的航空应急仿真培训平台。首期只完成气囊顶升 Web 模块，但从第一天起按“平台、仿真核心、三维客户端、数据、资产、设备”六类边界设计，后续增加灭火、VR 或力反馈时不重写基础平台。

首期必须完成

以 1 个明确子型号（建议资料最完整的机型）+ 1 个典型故障场景为对象，完整走通任务创建、三维训练、气囊布设、管路连接、充气顶升、错误判断、评分、事件记录、回放和云端部署。

首期明确不做

不在四个月内同时实现八机型、真实软体力学、高精度结构载荷计算、Unreal VR、实体力反馈设备、多节点微服务和复杂企业权限体系。

长期架构不打折

虽然首期不做 VR 与硬件，但接口、数据模型、资源规范和仿真核心必须按未来扩展要求设计，避免首期代码成为一次性原型。

质量目标

减少功能数量，不降低界面和交互完成度。网页应符合现代专业软件审美，三维训练要有清晰状态、连续动画、错误反馈和稳定帧率。

2. 最终技术架构

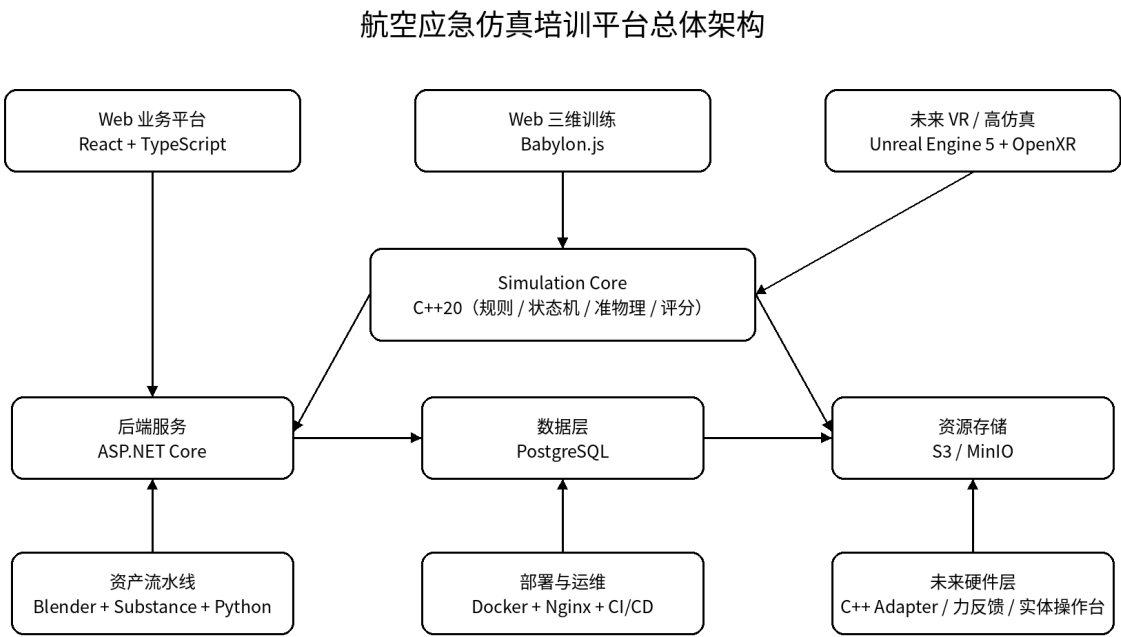


图 1 总体技术架构

模块	技术	职责
Web 业务平台	React + TypeScript	登录、任务、成绩、回放管理、教员后台、系统设置。
Web 三维训练	Babylon.js	当前气囊三维训练：飞机查看、点位、气囊拖放、管路、阀门、顶升动画。
仿真核心	C++20 + WebAssembly	训练状态机、规则、评分、准物理计算、事件模型。浏览器编译为WASM，未来 Unreal 原生调用同一逻辑。
后端	ASP.NET Core / C#	用户、任务、API、训练会话、成绩、回放索引、权限与文件服务。
数据库	PostgreSQL	用户、机型、场景、规则、事件、快照、成绩、版本信息。
三维资产	Blender + Substance 3D Painter	模型清理、减面、LOD、UV、材质、碰撞体、Web/VR 双版本资产。
自动化工具	Python	模型与配置批处理、数据校验、资源检查、自动测试辅助。

部署	Docker + Nginx	Web、API、数据库、静态资源统一部署；首期单机部署，后续按需拆分。
未来 VR	Unreal Engine 5 + OpenXR	灭火、烟雾、高沉浸训练、VR 交互和高级视觉。
未来硬件	C++ Adapter Layer	对接力反馈器、传感器、实体灭火器和操作台 SDK。

3. 模块划分与任务边界

模块	主要任务	首期输出	与其他模块关系
M0 需求与业务基线	冻结首期机型、故障、设备、流程、规则和验收终端。	需求基线、资料清单、规则表、验收矩阵	所有模块的输入；未冻结前不得大量制作模型或写死规则。
M1 Web 业务平台	完成登录、首页、任务、成绩、回放列表、教员后台基础页面。	可运行的 React Web 站点	调用 M4 API；进入训练时加载 M2。
M2 Web 三维训练	Babylon.js 场景、模型加载、镜头、点位、气囊、管路、阀门、顶升动画和 HUD。	可交互三维训练页面	读取 M6 资产与 M3 仿真状态；将用户操作提交给 M3/M4。
M3 Simulation Core	建立 C++ 状态机、规则引擎、准物理计算、评分、事件定义，并编译为 WASM。	Native/WASM 核心库 + 单元测试	M2 调用其进行本地即时判定；M4 保存权威记录。未来 Unreal 复用。
M4 后端服务	ASP.NET Core API、身份认证、任务、会话、成绩、事件、回放、资源索引。	可部署 API 服务	连接 M5；为 M1/M2/未来 Unreal 提供统一接口。
M5 数据库	设计关系表、JSONB 配置、事件与快照表、迁移脚本、备份策略。	PostgreSQL Schema + Migration	为 M4 提供持久化；保存版本以保证历史回放可复现。
M6 三维资产流水线	飞机与气囊模型整理、减面、LOD、碰撞、材质、坐标规范、导出 GLB。	首期机型 Web 资产包	直接供 M2 使用；同一源资产未来可导出 Unreal 版本。
M7 评分与回放	定义事件、错误、扣分、快照和时间轴恢复。	可复现训练过程的评分/回放系统	依赖 M3 规则、M4 API、M5 数据；M1 提供回放管理 UI。
M8 DevOps 与部署	Git 分支、CI、Docker、Nginx、HTTPS、日志、备份、环境配置。	开发/测试/生产三套可重复部署环境	从第 1 周即开始，不在最后一周临时部署。
M9 UI/UX 与视觉系统	Figma 设计系统、组件规范、网页 UI、三维 HUD、状态颜色和交互反馈。	统一设计规范和高完成度界面	贯穿 M1/M2，不作为最后阶段“补美化”。
M10 测试与验收	功能、规则、回放一致性、性	测试用例、缺陷清单、验收报	贯穿全程；每两周形成一次可

	能、兼容、部署恢复、异常测试。	告	验收版本。
M11 未来 VR/硬件预留	定义 Unreal 接口、OpenXR 设备抽象、IHapticDevice / ITrackingDevice 等契约。	接口文档和空实现/Mock	首期只留接口，不开发真实 VR 和力反馈。

4. 模块依赖与关键路径

整个首期项目的关键路径不是“先把所有页面做完，再做三维”，而是尽早建立一条可贯通的最小链路，然后逐步加深。推荐关键路径如下：

M0 需求基线 → M6 首个轻量模型 → M2 三维查看 → M3 状态机原型 → M4/M5 会话与记录 → M2 完整训练交互 → M7 评分回放 → M10 性能与验收 → M8 上线

M0 → M6：模型制作前必须确认具体子型号、需要观察的机腹部位和顶升区域，否则后期返工最大。

M6 → M2：Babylon.js 的交互代码依赖稳定的模型层级、坐标、碰撞体和命名规则。

M3 ↔ M2：M2 只负责表现和输入；位置是否正确、步骤是否允许、扣多少分由 M3 判定。

M4 ↔ M5：M4 不在代码里手写 SQL 业务逻辑，统一通过数据访问层和迁移管理数据库。

M3 → M7：评分与回放必须引用规则版本和事件定义，不能在回放页重新猜训练过程。

M9 → M1/M2：UI/UX 从第一周确定设计系统，避免网页和三维 HUD 各自长成两套产品。

M10 → 全模块：测试不是末期独立任务，每个模块都必须有可验收输出和回归检查。

5. 各模块详细任务清单

M0 需求与业务基线

任务	具体操作	输出	前置	直接后续	完成判定
M0-01 首期范围冻结	确定具体机型子型号、首期故障、训练模式、最终运行电脑/平板和浏览器。	项目范围一页纸；禁止使用“B737”这种过宽定义，必须具体到子型号。	无	M6、M2、M10	输出可被后续模块实际调用或通过对应验收用例；不以“代码已写”作为完成。
M0-02 资料收集与版本登记	整理飞机外形资料、顶升点、禁止区、气囊规格、管路和训练步骤；为每份资料记录来源、日期、版本和审核状态。	资料台账 + 版本号	M0-01	M3、M6	输出可被后续模块实际调用或通过对应验收用例；不以“代码已写”作为完成。
M0-03 标准流程拆解	把训练流程拆成“步骤、前置条件、允许操作、禁止操作、成功条件、错误、扣	训练流程表	M0-02	M3、M7	输出可被后续模块实际调用或通过对应验收用例；不以“代码已写”作为完成。

	分、纠正方式”。				
M0-04 验收矩阵	把每个功能转成可执行用例：输入、操作、预期结果、通过条件。	验收矩阵	M0-01~03	M10	输出可被后续模块实际调用或通过对应验收用例；不以“代码已写”作为完成。

M1 Web 业务平台

任务	具体操作	输出	前置	直接后续	完成判定
M1-01 前端工程初始化	建立 React + TypeScript 工程、路由、API Client、状态管理、环境变量、统一错误处理。	可启动项目骨架	M8-01	M1-02	输出可被后续模块实际调用或通过对应验收用例；不以“代码已写”作为完成。
M1-02 设计系统落地	按 Figma Token 建立颜色、字号、间距、按钮、表单、表格、对话框、状态提示组件。	UI Component Library	M9-01	所有 M1 页面	输出可被后续模块实际调用或通过对应验收用例；不以“代码已写”作为完成。
M1-03 登录与主导航	登录、退出、会话过期处理、主侧栏、顶部状态。	完整基础框架	M4-02	M1-04~07	输出可被后续模块实际调用或通过对应验收用例；不以“代码已写”作为完成。
M1-04 训练任务页	选择机型、故障、训练模式，展示任务说明和设备清单，创建训练会话。	Task Setup 页面	M4-03、M0	M2-01	输出可被后续模块实际调用或通过对应验收用例；不以“代码已写”作为完成。
M1-05 成绩页	显示总分、步骤得分、错误、时长和训练结果。	Score 页面	M7-03	验收	输出可被后续模块实际调用或通过对应验收用例；不以“代码已写”作为完成。
M1-06 回放入口	训练记录列表、筛选、查看详情、进入回放。	Replay List 页面	M4-05/M7	M7-04	输出可被后续模块实际调用或通过对应验收用例；不以“代码已写”作为完成。
M1-07 管理基础页	机型/场景/规则版本只读管理，首期不做复杂在线编辑器。	Admin 基础页	M4-06	后续扩展	输出可被后续模块实际调用或通过对应验收用例；不以“代码已写”作为完成。

M2 Web 三维训练

任务	具体操作	输出	前置	直接后续	完成判定
M2-01 Babylon 场景	创建 Engine、	空场景可稳定运行	M1-04、M6-01	M2-02	输出可被后续模块实

骨架	Scene、Camera、Light、环境地面、Resize、Loading、异常处理。				际调用或通过对应验收用例；不以“代码已写”作为完成。
M2-02 飞机模型加载	按任务下载当前GLB，建立节点索引、LOD、碰撞代理和资源释放。	首期飞机可旋转查看	M6-02	M2-03	输出可被后续模块实际调用或通过对应验收用例；不以“代码已写”作为完成。
M2-03 镜头与点位	实现轨道旋转、缩放、触控；显示顶升点、安全区、禁止区，可开关图层。	点位查看模式	M0-02	M2-04	输出可被后续模块实际调用或通过对应验收用例；不以“代码已写”作为完成。
M2-04 气囊选择与拖放	从设备栏生成气囊实例，地面/机腹投射拖动，吸附、旋转、撤销；提交 PlaceBag 事件。	可完成气囊铺设	M3-02	M2-05	输出可被后续模块实际调用或通过对应验收用例；不以“代码已写”作为完成。
M2-05 管路与阀门交互	选择软管端点、建立连接关系、显示正确/错误连接，阀门提供开闭状态。	管路操作可训练	M3-03	M2-06	输出可被后续模块实际调用或通过对应验收用例；不以“代码已写”作为完成。
M2-06 充气与顶升动画	根据 Simulation Core 输出的高度/姿态驱动气囊变形和飞机 Transform；动画采用平滑插值而非瞬移。	完整顶升视觉流程	M3-04、M6-03	M2-07	输出可被后续模块实际调用或通过对应验收用例；不以“代码已写”作为完成。
M2-07 三维 HUD	当前步骤、压力、同步差、错误、警告、下一步提示、暂停/退出。	统一训练 HUD	M9-03	M2-08	输出可被后续模块实际调用或通过对应验收用例；不以“代码已写”作为完成。
M2-08 Web 性能优化	按机型分包，启用 GLB 压缩/纹理压缩、LOD、实例化、资源释放；记录首载、内存、FPS。	性能报告	M6-04/M10-04	上线	输出可被后续模块实际调用或通过对应验收用例；不以“代码已写”作为完成。

M3 Simulation Core

任务	具体操作	输出	前置	直接后续	完成判定
M3-01 核心工程与跨平台构建	建立 CMake 工程，Native 单元测试；配置 Emscripten 生成 WASM；定义 JS/TS 调用边界。	Native + WASM Hello World	M8-01	M3-02	输出可被后续模块实际调用或通过对应验收用例；不以“代码已写”作为完成。

M3-02 训练状态机	实现 SessionState、StepState、前置条件、完成条件、回退和阻断。流程由配置驱动，不写死页面顺序。	可跑通虚拟流程	M0-03	M2-04/M7	输出可被后续模块实际调用或通过对应收用例；不以“代码已写”作为完成。
M3-03 规则引擎	RuleCode、Condition、Severity、Action、Penalty、Version；支持警告、阻断、扣分、失败。	规则判定 API	M0-03	M2/M7	输出可被后续模块实际调用或通过对应收用例；不以“代码已写”作为完成。
M3-04 气囊准物理模型	输入气囊位置、尺寸、充气级别和支撑状态；输出支撑高度、左右/前后高度差、教学用 Pitch/Roll、同步风险。	可解释仿真输出	M0-02	M2-06	输出可被后续模块实际调用或通过对应收用例；不以“代码已写”作为完成。
M3-05 事件模型	统一定义 BagPlaced、HoseConnected、ValveOpened、PressureChanged、StepCompleted、RuleTriggered 等事件及序列号。	事件契约	M3-02/03	M4/M7	输出可被后续模块实际调用或通过对应收用例；不以“代码已写”作为完成。
M3-06 未来接口预留	定义 SimulationModule、IHapticDevice、ITrackingDevice 等接口和 Mock，不实现真实设备。	接口文档 + Mock	M3-01	未来 Unreal/硬件	输出可被后续模块实际调用或通过对应收用例；不以“代码已写”作为完成。

M4 后端服务

任务	具体操作	输出	前置	直接后续	完成判定
M4-01 API 工程初始化	ASP.NET Core Web API、OpenAPI、日志、异常中间件、配置分环境、健康检查。	API 基础服务	M8-01	M4-02	输出可被后续模块实际调用或通过对应收用例；不以“代码已写”作为完成。

M4-02 认证与用户	首期账号密码、角色（管理员/教员/学员）、JWT 或 Cookie 会话；权限粒度保持简单。	登录 API	M5-01	M1-03	输出可被后续模块实际调用或通过对应收用例；不以“代码已写”作为完成。
M4-03 任务与会话	创建 TrainingTask、StartSession、FinishSession，固定记录 Asset/Scenario/Rule Version。	训练会话 API	M5-02	M1-04/M2	输出可被后续模块实际调用或通过对应收用例；不以“代码已写”作为完成。
M4-04 事件接收	按 SessionId + Sequence 接收事件，校验顺序，批量写入；关键事件同步生成快照。	事件写入 API	M3-05/M5-03	M7	输出可被后续模块实际调用或通过对应收用例；不以“代码已写”作为完成。
M4-05 成绩与回放查询	返回成绩明细、错误列表、事件时间线、快照索引。	查询 API	M7/M5	M1-05/06	输出可被后续模块实际调用或通过对应收用例；不以“代码已写”作为完成。
M4-06 资源与版本	返回可用机型、场景、规则版本和资源 URL；不把资源二进制塞数据库。	资源索引 API	M5/M6	M1/M2	输出可被后续模块实际调用或通过对应收用例；不以“代码已写”作为完成。

M5 数据库

任务	具体操作	输出	前置	直接后续	完成判定
M5-01 基础实体	User、Role、Aircraft、Scenario、Device、RuleSet、AssetVersion。	基础表迁移	M0	M4-02/06	输出可被后续模块实际调用或通过对应收用例；不以“代码已写”作为完成。
M5-02 训练实体	TrainingTask、TrainingSession、ScoreSummary、ErrorRecord。	训练表迁移	M5-01	M4-03	输出可被后续模块实际调用或通过对应收用例；不以“代码已写”作为完成。
M5-03 事件与快照	TrainingEvent 使用递增 Sequence；Payload 使用 JSONB；Snapshot 保存关键状态；建立 SessionId+Sequence 索引。	事件/快照表	M3-05	M4-04/M7	输出可被后续模块实际调用或通过对应收用例；不以“代码已写”作为完成。
M5-04 数据版本化	任何历史训练记录保留机型资源、场景、规则版本，不用“当	版本策略	M0-02	M7	输出可被后续模块实际调用或通过对应收用例；不以“代码

	前最新规则”解释旧数据。				已写”作为完成。
M5-05 备份与恢复	每日备份、保留周期、恢复脚本；测试环境至少实际恢复一次。	备份/恢复记录	M8	M10	输出可被后续模块实际调用或通过对验收用例；不以“代码已写”作为完成。

M6 三维资产流水线

任务	具体操作	输出	前置	直接后续	完成判定
M6-01 资产规范	统一米制、坐标轴、原点、命名、材质槽、碰撞代理、LOD命名、资源目录。	Asset Standard v1	M0-01	M6-02	输出可被后续模块实际调用或通过对验收用例；不以“代码已写”作为完成。
M6-02 飞机轻量化	删除驾驶舱/座椅/无关内饰和小零件；保留机腹、起落架、机翼、发动机外形；生成 LOD0/1/2、碰撞体。	Web 训练飞机资产	M0-02	M2-02	输出可被后续模块实际调用或通过对验收用例；不以“代码已写”作为完成。
M6-03 气囊/软管/阀门	依据设备尺寸自制训练模型；气囊至少制作 0/20/50/80/100% 形态或 Blend Shape。	设备资产包	M0-02	M2-04~06	输出可被后续模块实际调用或通过对验收用例；不以“代码已写”作为完成。
M6-04 Web 导出与压缩	导出 GLB；纹理按 2K/1K/512 分级；使用 KTX2/Basis 等 Web 友好压缩；建立资产清单和校验值。	可发布 Web Asset Pack	M6-02/03	M2-08	输出可被后续模块实际调用或通过对验收用例；不以“代码已写”作为完成。
M6-05 双端源资产保留	保留高质量 Source，不让 Web 减面版本成为唯一源文件；未来 Unreal 从 Source 重新导出。	Source/ Web 两套目录	M6-01	未来 VR	输出可被后续模块实际调用或通过对验收用例；不以“代码已写”作为完成。

M7 评分与回放

任务	具体操作	输出	前置	直接后续	完成判定
M7-01 评分模型	总分由必做步骤、顺序、参数区间、错误严重程度、时间等组成；所有扣分关联 RuleCode/Version。	评分规则表	M3-03	M7-03	输出可被后续模块实际调用或通过对验收用例；不以“代码已写”作为完成。

M7-02 快照策略	在步骤完成、关键状态变化或固定间隔保存 Snapshot；避免每帧保存。	快照策略	M4-04/M5-03	M7-04	输出可被后续模块实际调用或通过对验收用例；不以“代码已写”作为完成。
M7-03 成绩生成	训练结束按事件和规则版本计算并固化 ScoreSummary；生成步骤明细与错误清单。	成绩结果	M7-01	M1-05	输出可被后续模块实际调用或通过对验收用例；不以“代码已写”作为完成。
M7-04 回放引擎	从最近快照恢复状态，按 Sequence 重放事件；支持暂停、跳转、倍速和错误定位。	可操作回放	M7-02/M3-05	M1-06	输出可被后续模块实际调用或通过对验收用例；不以“代码已写”作为完成。

M8 DevOps 与部署

任务	具体操作	输出	前置	直接后续	完成判定
M8-01 代码仓与分支	建立 mono-repo 或清晰多仓；至少包含 web、api、sim-core、assets-tools、docs；启用 main/dev/feature 流程。	Git 仓库规范	无	所有开发	输出可被后续模块实际调用或通过对验收用例；不以“代码已写”作为完成。
M8-02 本地开发环境	Docker Compose 启动 PostgreSQL、API；前端和 WASM 可本地热更新；提供一键启动说明。	dev-compose	M8-01	开发效率	输出可被后续模块实际调用或通过对验收用例；不以“代码已写”作为完成。
M8-03 CI	提交后执行前端 build、后端 test、C++ test/WASM build、基础 lint。	CI Pipeline	M8-01	质量门禁	输出可被后续模块实际调用或通过对验收用例；不以“代码已写”作为完成。
M8-04 测试环境	Nginx + Web + API + PostgreSQL + 静态资产；HTTPS；日志滚动。	Test 环境	M8-02	M10	输出可被后续模块实际调用或通过对验收用例；不以“代码已写”作为完成。
M8-05 生产部署	首期单服务器容器化部署，资源文件单独目录/对象存储；配置备份、健康检查和回滚步骤。	Production Runbook	M10 通过	交付	输出可被后续模块实际调用或通过对验收用例；不以“代码已写”作为完成。

M9 UI/UX 与视觉系统

任务	具体操作	输出	前置	直接后续	完成判定
M9-01 Figma 设计系统	确定栅格、字体、颜色、按钮、输入、表格、卡片、提示、危险等级、图标。	Design Tokens	M0	M1/M2	输出可被后续模块实际调用或通过对验收用例；不以“代码已写”作为完成。
M9-02 关键页面原型	任务设置、三维训练、成绩、回放四页完成高保真原型并走通操作链。	高保真原型	M9-01	开发	输出可被后续模块实际调用或通过对验收用例；不以“代码已写”作为完成。
M9-03 三维 HUD 规范	HUD 不遮挡关键机腹区域；信息按“步骤/状态/警告/操作”分层；危险操作作用明确视觉和文字反馈。	HUD Spec	M9-01	M2-07	输出可被后续模块实际调用或通过对验收用例；不以“代码已写”作为完成。
M9-04 一致性检查	网页和 Babylon 训练页使用同一颜色、字号、术语、图标和状态文案。	UI QA 清单	M1/M2	M10	输出可被后续模块实际调用或通过对验收用例；不以“代码已写”作为完成。

M10 测试与验收

任务	具体操作	输出	前置	直接后续	完成判定
M10-01 单元测试	C++ 状态机、规则、评分必须有单元测试；后端核心服务有基础测试。	自动测试集	M3/M4	CI	输出可被后续模块实际调用或通过对验收用例；不以“代码已写”作为完成。
M10-02 端到端用例	从创建任务到成绩回放完整执行；覆盖正确流程、错误铺设、错误连接、超压、顺序错误等。	E2E 用例	M0-04	验收	输出可被后续模块实际调用或通过对验收用例；不以“代码已写”作为完成。
M10-03 回放一致性	随机抽取训练记录，比较事件序列、最终姿态、告警和得分。	一致性报告	M7	上线	输出可被后续模块实际调用或通过对验收用例；不以“代码已写”作为完成。
M10-04 性能与兼容	指定 PC/平板/浏览器测试首载、帧率、内存、触控和 2 小时稳定性；性能不达标先优化资产和渲染。	性能报告	M2-08	上线	输出可被后续模块实际调用或通过对验收用例；不以“代码已写”作为完成。
M10-05 部署恢复	验证重新部署、数据库恢复、会话失效、	运维验收记录	M8/M5	交付	输出可被后续模块实际调用或通过对验收

	无权访问、日志与备份。				收用例；不以“代码已写”作为完成。
--	-------------	--	--	--	-------------------

M11 未来 VR/硬件预留

任务	具体操作	输出	前置	直接后续	完成判定
M11-01 Unreal 适配契约	约定 Simulation Core Native API、事件字段、资源版本号和服务器 API，不开发实际场景。	接口说明	M3/M4	未来 Phase 3	输出可被后续模块实际调用或通过对应验收用例；不以“代码已写”作为完成。
M11-02 OpenXR/设备抽象	定义输入、位姿、振动/力反馈、设备状态接口；首期使用 Mock。	Device Contract	M3-06	未来硬件	输出可被后续模块实际调用或通过对应验收用例；不以“代码已写”作为完成。
M11-03 资产双版本策略	Web 只用轻量 GLB；保留 Source 资产用于未来 Unreal 高质量导入。	资产迁移规则	M6-05	未来 VR	输出可被后续模块实际调用或通过对应验收用例；不以“代码已写”作为完成。

6. 16 周详细时间线

执行方法：每两周形成一个可运行版本。任何阶段如果前置条件未完成，不要用临时代码绕过后继续堆功能。首期成功标准是“完整闭环 + 稳定 + 好用”，不是功能数量。

时间	具体任务	涉及模块	阶段交付物	依赖	里程碑/注意事项
第 1 周	冻结 M0-01~04；建立 Git/目录/分支；Figma 设计系统草案；ASP.NET Core/React/C++ CMake 工程初始化；PostgreSQL 本地容器。	M0,M8,M9,M3,M4,M5	需求基线、仓库、可一键启动开发环境、三端 Hello World	无	禁止进入大规模模型/交互开发，直到子型号和流程明确。
第 2 周	完成数据库基础实体、API 健康检查和登录骨架；Babylon 空场景；飞机资产规范；首个模型导入测试；Simulation Core Native/WASM 构建链。	M1,M2,M3,M4,M5,M6	登录壳、空三维页、WASM 被网页调用、模型导入样例	第 1 周	浏览器中能调用 WASM 并显示一个测试对象。
第 3 周	飞机模型正式轻量	M1,M2,M4,M6	任务选择→进入 B737	第 2 周	第一条端到端链路出

	化; Babylon 加载模型、镜头、LOD; Web 任务页雏形; API 提供机型/场景列表; 建立点位配置格式。		训练页; 飞机可旋转缩放		现, 哪怕还不能训练。
第 4 周	实现顶升点、安全区、禁止区图层; 训练会话 StartSession/FinishSession; Simulation Core 状态机最小版; CI 首次上线。	M2,M3,M4,M5,M8	创建任务→会话→三维点位查看→结束会话	第 3 周	形成 V0.1 可演示版本。
第 5 周	制作气囊/软管/阀门资产; 实现气囊选择、拖动、旋转、吸附; 核心规则判断允许区域; PlaceBag 事件入库。	M2,M3,M4,M6	气囊可布设且正确/错误位置有反馈	第 4 周	禁止把“是否正确”写在前端坐标 if 里。
第 6 周	完善铺设规则、撤销/重置; 管路端点与连接交互; 规则引擎支持警告/阻断/扣分; 事件序列和日志可查询。	M2,M3,M4,M5	铺设 + 管路连接闭环	第 5 周	形成 V0.2。
第 7 周	实现阀门、压力状态和气囊充气分级; Simulation Core 计算支撑高度; 气囊动画由核心输出驱动。	M2,M3,M6	0→20→50→80→100% 充气流程	第 6 周	动画和业务状态一致, 可重复。
第 8 周	实现飞机教学用 Pitch/Roll/Height; 同步偏差与风险; 完成“铺设→连接→充气→顶升→稳压→泄压→回收”最小流程。	M2,M3	首条完整训练流程	第 7 周	V0.3: 首期核心功能闭环。
第 9 周	建立评分模型、错误等级、RuleCode/Version; 完成成绩生成 API 和成绩页; 关键步骤保存快照。	M1,M3,M4,M5,M7	训练结束可生成可解释成绩	第 8 周	每个扣分都能追溯到具体事件和规则。
第 10 周	实现回放引擎: 快照恢复、事件顺序播放、暂停/跳转/倍速; 回放入口 UI。	M1,M2,M4,M7	训练可回放、错误可定位	第 9 周	形成 V0.4。
第 11 周	UI 高保真收口; 统一网页和三维 HUD; 增	M1,M2,M9	界面达到演示/交付完	第 10 周	不再使用开发态默认

	加 Loading、空状态、异常状态、确认对话框、操作提示；术语统一。		成度		组件和临时按钮。
第 12 周	模型/纹理/渲染优化；按需加载和资源释放；PC/目标平板第一轮性能测试；修复最重的 20% 性能问题。	M2,M6,M10	性能基线与优化版本	第 11 周	形成 V0.5；如果性能不达标，暂停加功能。
第 13 周	端到端异常测试：错误铺设、错误连接、超压、越序、重复操作、中断恢复；后端幂等与事件顺序检查。	M3,M4,M7,M10	核心缺陷清单清零一轮	第 12 周	关键规则 100% 有测试用例。
第 14 周	浏览器/触控/分辨率兼容；2 小时稳定性；回放一致性；数据库备份恢复；部署恢复演练。	M5,M8,M10	兼容/稳定/恢复报告	第 13 周	形成 RC1。
第 15 周	测试环境转生产配置；HTTPS、日志、备份、账号初始化；用户手册、部署手册、接口文档、资产台账。	M4,M5,M6,M8,M10	可交付生产包和文档	第 14 周	只修阻断验收的问题，不新增范围。
第 16 周	正式验收演练、缺陷缓冲、演示脚本、数据清理、版本冻结；输出 V1.0。	全模块	V1.0、测试报告、交付清单	第 15 周	首期结束；进入机型复制或 VR 预研。

7. 里程碑与验收门槛

里程碑	必须具备	门槛
V0.1（第 4 周）	任务创建、三维飞机查看、点位展示、训练会话可记录。	架构已贯通，不再允许大改技术栈。
V0.2（第 6 周）	气囊铺设、管路连接、规则判定、事件入库。	三维表现与规则彻底分离。
V0.3（第 8 周）	完整顶升流程可从开始走到结束。	首期核心价值成立；之后重点转评分、回放和质量。
V0.4（第 10 周）	成绩、错误、事件、快照和回放完成。	系统从“三维演示”升级为“培训系统”。
V0.5（第 12 周）	UI 收口，目标终端性能达到预期。	冻结大功能。

RC1 (第 14 周)	功能、性能、兼容、恢复基本通过。	只允许修 Bug。
V1.0 (第 16 周)	部署、文档、验收闭环。	首期正式交付。

8. 一条真实训练链路如何跨模块运行

- 1. 创建任务：**React 选择机型/故障/模式 → ASP.NET Core 创建 TrainingTask/Session → 数据库固定 ScenarioVersion/RuleVersion/AssetVersion。
- 2. 加载场景：**Babylon.js 根据 API 返回的资源索引，仅下载当前机型 GLB、气囊和场景配置。
- 3. 用户铺设气囊：**Babylon.js 只读取拖动结果并生成 PlaceBag 输入；Simulation Core 判断是否进入允许区并返回状态。
- 4. 记录事件：**前端把规范化事件提交 API；服务器按 SessionId/Sequence 入库。
- 5. 连接与充气：**连接、阀门、压力变化都作为事件；Simulation Core 更新状态并计算支撑高度、Pitch/Roll 和风险。
- 6. 三维表现：**Babylon.js 不自行决定物理结果，只根据核心输出平滑驱动气囊和飞机动画。
- 7. 评分：**RuleTriggered 事件携带 RuleCode/Version/Penalty；训练结束生成不可变成绩明细。
- 8. 回放：**读取最近 Snapshot，再按 Sequence 重放事件；画面、告警和得分能够复现。
- 9. 未来 VR：**Unreal 不重新发明规则，只调用 Simulation Core Native 版本和同一后端 API，实现相同事件与评分体系。

9. 建议代码与资源目录

```

aviation-training/
├── apps/
│   ├── web/           # React + TypeScript
│   ├── api/           # ASP.NET Core
│   └── web-sim/       # Babylon.js 训练模块
├── core/
│   └── simulation-core/ # C++20 / CMake / WASM
├── packages/
│   ├── contracts/     # 事件、DTO、Schema
│   └── design-tokens/  # UI 设计 Token
├── assets/
│   ├── source/        # 高质量源资产
│   ├── web/           # GLB/KTX2/LOD
│   └── metadata/       # 点位/区域/版本
├── tools/             # Python 资产与数据工具
└── deploy/            # Docker/Nginx/环境配置

```

└─ tests/

E2E/回放一致性/性能脚本

└─ docs/

需求、接口、数据库、资产规范

10. 首期必须统一的关键数据契约

对象	首期核心字段
TrainingSession	SessionId、TaskId、AircraftVersion、ScenarioVersion、RuleVersion、StartAt、Status
TrainingEvent	SessionId、Sequence、OccurredAt、Actor、ObjectId、Action、Parameters、StateBefore、StateAfter、TriggeredRules、ScoreDelta
Snapshot	SessionId、Sequence、AircraftState、BagStates、HoseStates、ValveStates、StepState
Rule	RuleCode、AppliesTo、Condition、Severity、Action、Penalty、Version
AircraftPackage	ModelUrl、LOD、LiftingPoints、安全区、禁止区、碰撞代理、资源版本
SimulationState	Step、Pitch、Roll、Height、PressureStates、SyncDeviation、RiskLevel

11. 首期之后的扩展路线

阶段	主要任务	复用内容
Phase 2：多机型/多场景复制	在 V1.0 架构不变的前提下新增机型包、场景包和规则版本。每新增一个机型先做资产轻量化与业务审核，再进入自动化回归。	平台、评分、回放、账号不重写。
Phase 3：Unreal VR 灭火模块	创建独立 Unreal 客户端，接入 OpenXR；调用 Simulation Core Native；实现火焰/烟雾/灭火剂视觉、VR 抓取和训练事件。	后端/API/规则/事件体系继续复用。
Phase 4：力反馈与实体设备	通过 C++ Device Adapter 对接厂商 SDK 或自研控制器；先 Mock，再真实设备；控制频率与安全边界独立于 Web。	只新增设备适配层和设备配置。
Phase 5：更多应急训练模块	疏散、救援设备、机体扶正、地面保障等均实现为新的 SimulationModule + 场景资源 + 规则包。	不复制整套系统。

12. 主要风险与处理顺序

风险	影响	处理
资料未冻结	模型、顶升点和规则返工	第 1 周必须冻结首期对象；所有外部数据带版本。
Web 三维性能不足	加载慢、平板崩溃	先减模型/纹理/Draw Call/内存，再考虑改架构；第 12 周前必须实测目标设备。
规则写在 UI	未来 Web/VR 结果不一致	所有判定进入 Simulation Core；UI 只展示。
回放靠录屏	无法解释评分与状态	事件 + 快照；所有事件带 Sequence 和版本。
首期范围失控	四个月做不完	第 8 周后冻结大功能；八机型/VR/力反馈不进入首期。
资产版权不清	无法交付	资产台账保存许可证、购买凭证、修改记录和适用范围。
最后才部署	环境问题集中爆发	第 2 周本地 Docker，第 4 周 CI，第 8~10 周测试环境常态化。

13. V1.0 最终交付清单

类别	交付内容
软件	React Web 平台、Babylon.js 三维训练模块、Simulation Core WASM、ASP.NET Core API、数据库迁移、Docker/Nginx 配置。
功能	任务设置、三维点位、气囊铺设、管路连接、阀门/压力、同步顶升、稳压/泄压/回收、评分、训练记录、回放。
资产	首期飞机 Web 模型、气囊/软管/阀门、LOD、碰撞体、材质、点位/区域配置、资产台账。
测试	单元测试、端到端用例、回放一致性、性能/兼容、稳定性、部署恢复报告。
文档	需求基线、总体架构、接口契约、数据库设计、资产规范、部署手册、用户手册、验收矩阵。
扩展预留	SimulationModule、Unreal Native API、OpenXR/力反馈设备接口、Source 级高质量资产保留。

14. 执行结论

首期四个月应把“气囊顶升”做成真正可训练、可评分、可回放、可部署的完整模块，而不是快速堆出多个机型。长期最重要的不是提前实现 VR，而是从首期就保证以下边界正确：Web 负责呈现，Simulation Core 负责规则和仿真，ASP.NET Core 负责平台与权威数据，资产按版本包管理，设备通过抽象接口接入。

项目判断标准：第 16 周如果只有一架飞机，但完整流程稳定、界面成熟、评分可解释、回放可复现、性能达标并且后续机型/VR 能在现有边界上扩展，则项目成功；如果有很多模型和页面，但规则写死、数据不可追溯、回放不可复现，则不算完成。